

Assignment 11

In this exercise sheet, we look at the application and some approximation properties of one-step methods.

Graded exercise

Exercise 37 (Explicit integrators for a predator-prey model)

This exercise is devoted to coding two explicit time stepping methods and applying them to a Lotka-Volterra system. For learning purposes, here you are not allowed to use ODE integrators from the `scipy` library. However, for future use, be aware that `scipy.integrate` contains many routines to integrate numerically ODEs (often with a so-called adaptive method).

- (a) Write a PYTHON function `explicit_Euler(f,t0,u0,tau,N)` that, given in input a function handle `f` to the right-hand side, the initial time `t0` and state `u0`, the time step size `tau` and the number `N` of time steps, solves the ODE $u'(t) = f(t, u), t > t_0$ with $u(t_0) = u_0$ using the *explicit Euler method*

$$u_n = u_{n-1} + \tau f(t_{n-1}, u_{n-1}).$$

The routine should return the solution at all time steps, namely $u_0, \dots, u_N$. Write the routine in such a way that it works for a system of ODEs, that is in the case that $u : [t_0, \infty) \rightarrow \mathbb{R}^m$ with m possibly greater than one.

- (b) Write a routine `modified_Euler(f,t0,u0,tau,N)` that works similarly to the previous one but now computing the numerical solution with the *modified Euler method* (aka explicit midpoint rule)

$$\begin{aligned} u_{n-\frac{1}{2}} &= u_{n-1} + \frac{\tau}{2} f(t_{n-1}, u_{n-1}), \\ u_n &= u_{n-1} + \tau f(t_{n-\frac{1}{2}}, u_{n-\frac{1}{2}}), \end{aligned}$$

where $t_{n-\frac{1}{2}} = (n - \frac{1}{2}) \tau + t_0$.

- (c) We now consider a Lotka-Volterra system that models the population dynamics of an ecosystem with predators y and preys x . The time evolution of the number of predators and preys is described by the following system of ODEs:

$$\begin{aligned}x'(t) &= \lambda_1 x \left(1 - \frac{y}{y_e}\right), \\y'(t) &= \lambda_2 y \left(\frac{x}{x_e} - 1\right),\end{aligned}$$

for $t \in (t_0, T]$ and with some initial conditions $x(t_0) = x_0$, $y(t_0) = y_0$. We take the following values for the parameters: $[t_0, T] = [0, 20]$, $(x_0, y_0) = (10, 5)$, $(x_e, y_e) = (15, 10)$ and $(\lambda_1, \lambda_2) = (1, 0.5)$. Write a PYTHON script that, for each time step size $\tau_i = (T - t_0)0.0125 \cdot 2^{-i}$ with $i = 0, \dots, 3$, computes the numerical solutions to the ODE using the explicit Euler and modified Euler methods, and plots them in the same phase plane (x, y) (to clarify: one plot per time step size, and, in each plot, the curves corresponding to the different numerical solutions). Remember to hand in all the plots! The exact solution of the system is periodic, that is it looks like a closed orbit in the phase plane. Which of the two numerical schemes approximates this behavior best?

Other exercises

Exercise 38 (Autonomization)

We consider the non-autonomous initial value problem

$$\begin{aligned}u'(t) &= f(t, u(t)), \quad t > t_0, \\u(t_0) &= u_0.\end{aligned}\tag{1}$$

Introducing the variable $w(t) = (w_1(t), w_2(t))^T$, we consider the associated autonomized system

$$w'(t) = \begin{pmatrix} f(w_2(t), w_1(t)) \\ 1 \end{pmatrix} := F(w), \quad w(t_0) = \begin{pmatrix} u_0 \\ t_0 \end{pmatrix} := w^0.\tag{2}$$

- (a) Assume that (1) and (2) admit a unique solution. Show that $w(t) = (u(t), t)^T$.
- (b) A Runge-Kutta method is called *invariant under autonomization* (or autonomization-invariant) if its application to (1) and (2) gives the same numerical solution, that is, if $w_n = (u_n, t_n)^T$. Show that, if the coefficients of a Runge-Kutta method satisfy

$$\sum_{j=1}^s b_j = 1, \quad \text{and} \quad \sum_{j=1}^s a_{ij} = c_i, \quad \text{for } i = 1, \dots, s,$$

then the method is invariant under autonomization.